



PDF Download  
3578357.3589458.pdf  
18 March 2026  
Total Citations: 5  
Total Downloads: 299

 Latest updates: <https://dl.acm.org/doi/10.1145/3578357.3589458>

RESEARCH-ARTICLE

## Browser-in-the-Middle - Evaluation of a modern approach to phishing

JONAS TZSCHOPPE, Nuremberg Institute of Technology, Nurnberg, Bayern, Germany

HANS LÖHR, Nuremberg Institute of Technology, Nurnberg, Bayern, Germany

Open Access Support provided by:

Nuremberg Institute of Technology

Published: 08 May 2023

[Citation in BibTeX format](#)

EUROSEC '23: 16th European Workshop  
on System Security  
May 8, 2023  
Rome, Italy

Conference Sponsors:  
SIGOPS

# Browser-in-the-Middle - Evaluation of a modern approach to phishing

Jonas Tzschope

tzschopejo72172@th-nuernberg.de

Technische Hochschule Nürnberg Georg Simon Ohm  
Nürnberg, Germany

Hans Löhr

hans.loehr@th-nuernberg.de

Technische Hochschule Nürnberg Georg Simon Ohm  
Nürnberg, Germany

## ABSTRACT

This paper examines the phishing technique Browser-in-the-Middle and its practical implications in the context of logins protected by multi-factor authentication. We implement and analyze Browser-in-the-Middle (BitM) attacks, evaluate them and discuss suitable measures for mitigation. To facilitate a thorough analysis, we implement two variants of BitM by using two different technology stacks and compare them to a conventional phishing system based on a proxy. To evaluate BitM attacks, we test our implementations on a number of popular websites. Our results show that in practice BitM attacks are currently highly capable of stealing login information protected by more than one factor, since the difficulty to detect such an attack appears to be greater when using BitM than comparable techniques. Therefore, we propose a new entry for BitM in the Common Attack Patterns Enumeration and Classification (CAPEC). The high effectiveness of the attack technique is limited by mitigation methods such as the use of resistant factors for two-sided authentication. We conclude that BitM attacks can potentially be used for highly effective targeted phishing, but they are unlikely to scale well enough for large-scale phishing attacks aiming at a broad variety of users.

## KEYWORDS

Web Security, Phishing, MFA, Browser-in-the-Middle attacks, BitM

### ACM Reference Format:

Jonas Tzschope and Hans Löhr. 2023. Browser-in-the-Middle - Evaluation of a modern approach to phishing. In *European Workshop on System Security (EUROSEC '23)*, May 8, 2023, Rome, Italy. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3578357.3589458>

## 1 INTRODUCTION

Phishing attacks present a long-standing, yet unsolved, problem on the internet and attackers continuously devise new techniques to thwart mitigation efforts [3]. Recently, Tommasi et al. [19] introduced *Browser-in-the-Middle (BitM)*, a novel phishing method

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

*EUROSEC '23*, May 8, 2023, Rome, Italy

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0085-9/23/05...\$15.00

<https://doi.org/10.1145/3578357.3589458>

exploiting browser-based remote desktop connections that has not received a lot of scrutiny so far.

With this paper, we aim towards closing this gap by investigating the practical “application” of BitM for phishing attacks. Moreover, we want to raise awareness among security practitioners about potential dangers arising from this little-known attack method.

Towards these goals, we implement variants of BitM and compare them to a classical proxy-based method. We use the Common Attack Pattern Enumeration and Classification (CAPEC) framework [7] to make knowledge about BitM available to a broader audience of security practitioners.

*Contribution.* Specifically, our contributions include:

- **Implementation:** We implement two variants of BitM using open source software.
- **Evaluation:** We evaluate practical possibilities of the attack technique using frequently targeted web services.
- **Mitigation:** We derive from our tests forms for mitigation proven to be effective against such attacks.
- **Attack Pattern:** We summarize previous and new knowledge in a newly created CAPEC entry that got accepted by MITRE.

*Structure.* In Section 2, we briefly summarize related work, before we explain the necessary background in Section 3. We present our architecture and implementation in Section 4 and the evaluation of our experiments in Section 5. After discussing the results in Section 6, we conclude the paper in Section 7 with a brief outlook on potential future work.

## 2 RELATED WORK

This paper builds on the work of [19] who originally introduced BitM as a new form of man-in-the-middle attacks (cf. Sec. 3). The security researcher with the alias Mr. D0x particularly inspired our study, especially for the implementation of BitM, with his blog article [14]. He suggested the use of BitM in a phishing campaign and shared best-practices for doing so. Apart from that, there is currently no scientific publication explicitly studying Browser-in-the-Middle attacks.

Adam Oest et al. 2020[16] recently analyzed large datasets of real phishing campaigns to give insights about the end-to-end life cycle and effectiveness of phishing attacks at scale. Addressed scalability issues of BitM become clear when their work is taken into account.

We used Common Attack Pattern Enumeration and Classification (CAPEC)[7], a dictionary of known attack patterns which is

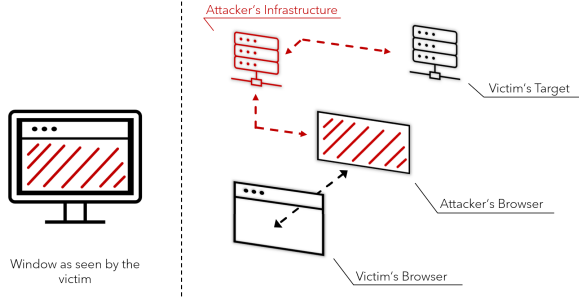
curated by The MITRE Corporation, to record the results and findings of our work in an accessible form for practitioners.

Man-in-the-Browser (MitB) attacks [5] are a different form of Man-in-the-Middle attacks and not to be confused with BitM attacks: For MitB, the attacker needs to compromise the victim's browser, whereas BitM works completely without compromising the client system.

Dhamija et al. [4] investigate the effectiveness of conventional phishing sites to visually deceive victims, but they do not study BitM attacks. Our work, however, focuses on the technical potential of BitM attacks.

### 3 BACKGROUND

Recently, Tommasi et al. [19] published a creative new approach to the well-known Man-in-the-Middle attack called Browser-in-the-Middle (BitM).



**Figure 1: Overview of a Browser-in-the-Middle attack.**

Fig. 1 illustrates a Browser-in-the-Middle attack (BitM): An adversary exploits the inherent functionalities of a web browser, to establish an unnoticed remote desktop connection from the victim's browser to a browser hosted by the adversary. As [19] pointed out, a BitM attack, unlike other attacks such as the Man-in-the-Browser attack, does not need the victim to install malicious software. Instead, a BitM attack does require the adversary to deploy a web client the victim can access, to enable it to connect to the attacker's system. After the victim's browser establishes the remote-desktop connection through the client hosted on the adversary's system, the victim interacts with the actual targeted application through the browser of the attacker in a normal manner. In a phishing scenario, the victim's target could be an online banking website for instance.

- All actions done in the open window by the victim are actually done on the machine of the adversary.
- Any victim-authenticated sessions are then available for full use on the machine of the adversary. All entered data such as passwords and usernames can be logged by the adversary.
- The content displayed on the victim's browser can be altered arbitrarily.
- Both integrity and confidentiality of the data is being compromised.

In this study we take the approach of [14] to examine to what extent Browser-in-the-Middle is suitable in practice as a technique for phishing in order to attack logins protected by multi-factor

authentication and what relevance this method has in practice compared to previous techniques of phishing. We used the findings to formulate viable ways to mitigate such an attack and documented the results in a new entry of CAPEC.

### 4 ARCHITECTURE AND IMPLEMENTATION

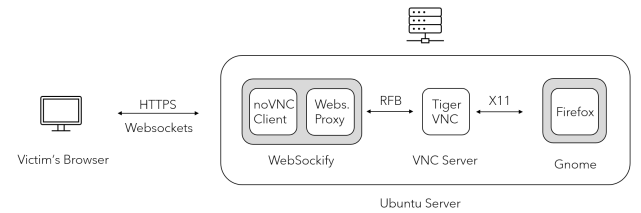
In order to evaluate the performance, practicability, and other properties of Browser-in-the-Middle, we implemented the technique with two different technology stacks. In this paper, we refer to them as RFB and RDP variants, based on the remote-desktop protocol being used.

**Table 1: Soft- & Hardware components of the BitM variants we implemented**

| Used Software  |             | Used Hardware |                |              |
|----------------|-------------|---------------|----------------|--------------|
| Description    | Version     | Description   | Windows Server | Linux Server |
| Windows Server | 2019 LTSC   | CPU           | Intel i7 6700  | AMD R5 3600  |
| Ubuntu Server  | 22.04.1 LTS | CPU Clock     | 3.4 GHz        | 3.6 GHz      |
| Gnome          | 42.3        | RAM           | 16 GB          | 64 GB        |
| TigerVNC       | 1.12.0      | Network       | 1 Gbit/s       | 1 Gbit/s     |
| noVNC          | 1.3.0       | Location      | Berlin         | Frankfurt    |
| Tomcat         | 9.0.65      |               |                |              |
| Guacamole      | 1.4.0       |               |                |              |
| Firefox        | 103.0.2     |               |                |              |

#### 4.1 Remote Frame Buffer Variant

The BitM variant which we implemented based on RFB uses the virtual desktop infrastructure VNC that is popular in Linux environments. VNC is built on the Remote Framebuffer Protocol [17]. It is on the one hand used to send user input like mouse movement and keystrokes to the remote system and on the other hand to receive images to be displayed on the local system. There exists a huge variety of mostly inter-compatible VNC servers and clients called viewers. As shown in Fig. 2, we use the VNC viewer noVNC and the VNC server Tiger VNC, which both are open source. This largely corresponds to the VNC software used by [19] for their proof-of-concept and the implementation proposed by [14].



**Figure 2: Illustration of the implementation of BitM in the RFB variant**

As already mentioned, the victim initiates a remote desktop connection by clicking on a link provided by the attacker. The victim's browser will then establish a session using HTTPS. That session is then upgraded to a secured WebSocket connection. The web client is hosted on the lightweight webserver WebSockify that

translates the traffic coming from the victim's browser via HTTPS / WSS to the RFB-Protocol, that the Websocket proxy is using to communicate to the actual VNC Server Tiger VNC. After the start of TigerVNC, a virtual display, the so-called X server gets created, which is then used by the Linux user environment Gnome that contains the Firefox browser running in full-screen mode.

Since the original intent of the software used in our tests differs a lot from phishing, we did several changes to the software to make the appearance and usage of the web client as inconspicuous as possible.

Fig. 3 shows the native appearance of the noVNC client in its unmodified state. Necessary changes have been highlighted red, e.g., the favicon that we replaced with the one of the website being phished for. Minor changes to the JS Code removed the appendix "noVNC" so the title of the targeted page could be inserted. Beside that Fig.3 shows UI elements like the menu on the left and the info text on top which we removed as well as animations during the connection process. We removed those by changes to the JS and CSS of the web client.

To avoid warnings for a unencrypted connection, we registered a placeholder domain "bitmiddle.de" and created an SSL-Certificate for it, to let a victim's browser establish the connection via HTTPS, this way increasing the trust of a potential victim.

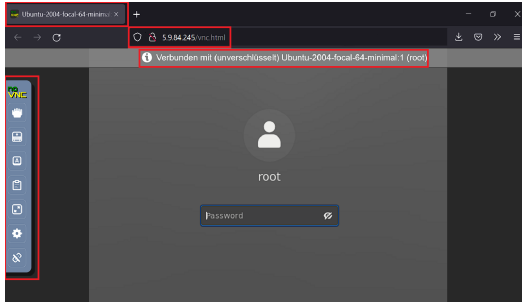


Figure 3: User view of the unmodified noVNC client

## 4.2 Remote Desktop Protocol Variant

The second variant of BitM we implemented differs from the first variant beside the used protocol FreeRDP [8] mostly in the architecture of the attacker's platform. In the RDP variant the web client and the user environment run on two separate systems. We used a guacamole web client [10] that is hosted on a tomcat webserver, which itself is running on a Ubuntu Linux server. It is acting as a gateway and is distributing one or more victims to one or more windows servers that contain the running instances of the shared web browsers.

The original use case of Guacamole is to provide access to remote computers via a web client using various protocols, such as RDP, VNC, but also SSH. After user log in to Guacamole with their ID and password they have point-and-click access to the connections assigned to them.

A mechanism, where a connection must be selected by a logged-in user, is unsuitable though for using it as a phishing tool, that's why more extensive changes to guacamole were necessary than to noVNC in the 1st variant.

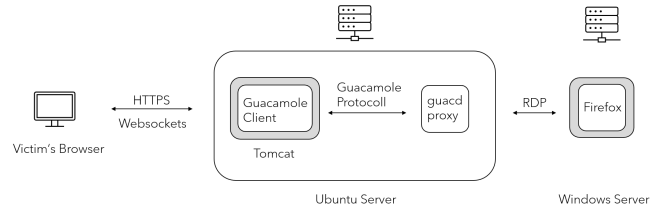


Figure 4: Illustration of the implementation of BitM in the RDP variant

Guacamole supports extensions written in Java that add or change functionality of the application. Installing the extension NoAuth [15] replaces the default authentication mechanism of Guacamole. As a result, logging in to the application is no longer required. This enables a victim to immediately establish a remote desktop connection after clicking on a link.

In the default implementation of Guacamole, a thumbnail version of the transferred screen is displayed in the tab instead of a static image, which gives the user an overview of open connections and their contents. For this purpose, an icon gets generated from the visible content and inserted in the tab as favicon at intervals of a few seconds using JavaScript. We changed this mechanism to only display the favicon of the targeted web service as a static image.

Moreover, changes were necessary to prevent obvious information from being shared with the user during initialization and active connection. We removed all unnecessary elements being displayed and concealed those that originated during the already active connection, i.e., the login information from the used windows environment, by changes to CSS, JS and HTML.

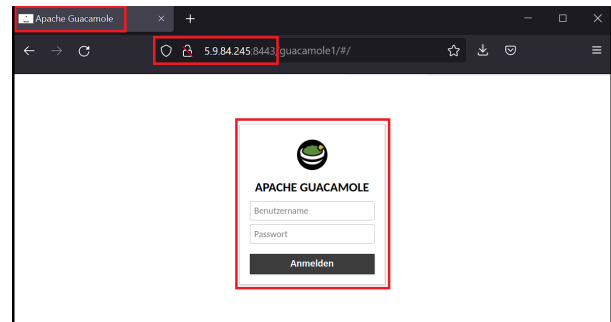


Figure 5: User view of the unmodified Guacamole client

## 5 EVALUATION

### 5.1 Test Setup

For the tests, we chose a selection of globally relevant targets to derive conclusions about the performance of BitM. We chose all targets from three collections of web services targeted the most in phishing campaigns [1, 18, 20]. All of them had to meet the requirement of implementing a form of MFA to secure the login and being available in germany, where we conducted our study.

Beside the two variants of a BitM attack, we implemented a traditional phishing system using a proxy based on the open-source phishing tool Evilginx2[9]. Using this third implementation, we were able to set the performance of BitM in relation to comparable types of phishing. We ran tests in different scenarios as shown in Tab. 2. This way we could cover most of the technical conditions under which a potential victim would come to face such an attack.

**Table 2: Scenarios for the test clients**

|                                            | Scenario 1                              | Scenario 2                              | Scenario 3                            | Scenario 4            |
|--------------------------------------------|-----------------------------------------|-----------------------------------------|---------------------------------------|-----------------------|
| <b>Platform</b>                            | Desktop                                 | Desktop                                 | Desktop                               | Mobile                |
| <b>OS</b>                                  | Windows 11                              | Ubuntu 16.4.1                           | macOS 12                              | Android 12            |
| <b>Display Resolution and Refresh Rate</b> | 1920 x 1080 @ 60 Hz                     | 2560 x 1440 @ 144 Hz                    | 3024x1964 @ 120 Hz                    | 1080 x 2400 @ 60 Hz   |
| <b>Used Browser</b>                        | Chrome 104.0.5112.81<br>& Firefox 105.0 | Chrome 104.0.5112.81<br>& Firefox 105.0 | Chrome 104.0.5112.81<br>& Safari 15.3 | Chrome 105.0.5195.136 |

## 5.2 Results

As shown in Tab. 3, an attack using one of the two variants of BitM could be successfully carried out in 12 out of 13 cases, while an attack using the proxy Evilginx2 has been successful in two out of six cases. For all entries in Tab. 3 marked with “success”, the

**Table 3: Results of phishing by means of the variant used by target.**

| Target | Used MFA             | BitM - RFB             | BitM - RDP             | MitM - Evilginx2  |
|--------|----------------------|------------------------|------------------------|-------------------|
| #1     | WebAuthn             | Connection denied      | Connection denied      | Connection denied |
| #2     | Push-Message via App | Success                | Success                | Failure           |
| #3     | OTP via SMS          | Success                | Success                | Success           |
| #4     | OTP via SMS          | Success                | Success                | Failure           |
| #5     | TOTP                 | Success                | Success                | Success           |
| #6     | OTP via SMS          | Success                | Success                | No phishlet       |
| #7     | WebAuthn + OTP       | Success after Fallback | Success after Fallback | Failure           |
| #8     | OTP via SMS          | Success                | Success                | No phishlet       |
| #9     | OTP via SMS          | Success                | Success                | Failure           |
| #10    | Push-Message via App | Success                | Success                | No phishlet       |
| #11    | TOTP                 | Success                | Success                | No phishlet       |
| #12    | Push-Message via App | Success                | Success                | No phishlet       |
| #13    | WebAuthn + TOTP      | Success after Fallback | Success after Fallback | Failure           |

transmitted user ID and password could be intercepted during the attack and the current session could be taken over. This was also the case for all entries marked with “success after fallback”, but only after the initial authentication via hardware key failed and a fallback factor has been offered to the user instead. In both cases

that fallback factor has been a form of one-time passwords. This also means that without the victim actively selecting the fallback factor, a successful Browser-in-the-Middle attack could be prevented in those cases by using hardware security keys.

If a tested service offered the use of WebAuthn as a second factor, this type of MFA has been selected. In those cases WebAuthn was realized via a hardware key.

Phishing with BitM worked for all targets in this test, except for one. In that case, any connection attempts from the used servers to the target were generally denied, which prevented any successful authentication. This was the case both during the BitM attack and natively, without any attack on the authentication.

If using the proxy failed, the connection was either rejected with an error message or has been reset. For the remaining entries marked with “no phishlet” we did not test the proxy-based phishing attack, because no public phishlet, that is a pre-configuration for one specific target, was available.

Phishing of mobile users using any of the two implementations of BitM proved to be unfeasible, due to insufficient optimized clients supporting remote desktop connections. Hence, Tab. 3 only shows the results of the test for each targeted web service in the desktop scenarios.

**Table 4: Average resource usage of the phishing variant on the host system**

| Target         | RAM Usage   | Network Usage |                   |
|----------------|-------------|---------------|-------------------|
|                |             | Send in MByte | Received in MByte |
| BitM RFB       | ~ 700 MByte | 12,759        | 0,462             |
| BitM RDP       | ~ 400 MByte | 17,935        | 0,791             |
| MitM Evilginx2 | ~ 20 MByte  | 0,360         | 0,207             |

Regarding the different test scenarios, the used OS as well as the used resolution and refresh rate had no impact on the results. The same holds for the display resolution of the victim’s device, since both implementations of BitM support dynamic resizing of the transmitted display. The refresh rate was irrelevant if local rendering of the cursor was being used.

For the RDP variant of BitM it did make a difference which browser the victim used, since the guacamole web client supports the use of the clipboard while the noVNC client doesn’t. In the test only Chrome allowed the clipboard to be shared with a web application but only after the user gave active permission for it. Safari and Firefox did not support any sharing of the clipboard using both clients.

The most apparent difference of the three implemented types of phishing beside the success rate is the amount of resources needed as shown in Tab. 4. Compared to proxy-based phishing, the test shows that both BitM implementation required significantly more resources, both in network and memory usage. While showing only a minor difference in network usage, the BitM variants via RFB and RDP differ strongly in their memory usage, because of the aforementioned separation of the user environment and the hosting of the web client. We cannot confirm the hypothesis raised

by [19] that network connectivity becomes a bottleneck when the method is scaled up. The results of our tests show a lower resource requirement per victim to the network than to the main memory, which is accordingly becoming a bottleneck first, depending on the resources available.

## 6 DISCUSSION

### 6.1 Effectiveness

Browser-in-the-Middle has proven to be very effective for attacking authentication secured by multiple factors. Almost all currently used variants of MFA can be bypassed. Only authentications using a security key as second factor were not vulnerable to the attack, in principle. However, in the test BitM was effective even where security keys were used, since fallback methods vulnerable to BitM were offered in each of the tested cases after the initial authentication failed. If a user then continues the login procedure, the attack is successful.

Especially in comparison with other methods of phishing, BitM is effective, since it seems to be harder for the targeted web services to detect such an attack. During phishing in traditional MitM attacks, HTML, JS, and CSS is passed over a proxy to the victim's browser. This gives the targeted web service the possibility to use, e.g., JS to get information about any communication participants in between, which might be used to detect man-in-the-middle attacks. But in a BitM attack, the actual HTML, JS, and CSS only reach the attacker's browser and only visual information is sent to the victim.

Also, when using BitM, the attacker does not have to steal a session, e.g., by using session hijacking, as in other types of phishing, because the only authenticated session is the one of the attacker's browser. From the perspective of the target website, this session is indistinguishable from a normal session.

Some countermeasures like the heuristic approach of capturing the structure of a phishing page and its content, such as images, text, and icons, and comparing these, with a legitimate website as seen in [12], do not work for BitM attacks. Because here the web page visited by the victim only contains HTML, CSS, and JS necessary to display the desktop transmission. Even if the displayed website is visually almost equal to the original, there is no similarity from the machine's point of view.

With the variants of BitM implemented in this work, effective phishing of victims using mobile devices was not possible. However, as already mentioned by [19], it cannot be ruled out that web clients with appropriate support for remote desktop connections will be developed to target mobile devices, too.

### 6.2 Practicability

To implement automated real-time phishing similar to proxy-based attacks, the attacker must study the targeted victim services in advance. This procedure must be repeated for each new target. For some websites, the required information could already be publicly available, as is the case with the Evilginx phishlets mentioned earlier. However, as our tests show, this information must also be kept up to date, as target pages react to attacks and protect themselves by making changes.

With BitM, the structure and branching of a target site's requests do not need to be known before the attack. Compared to

proxy-based phishing, the high site-specific overhead to handle authentication requests and study their structure for multiple targets shifts to a high initial overhead required for the implementation of the infrastructure of the attack. Once BitM is implemented, the system can easily be deployed to any number of targeted web services, because the remaining effort per target is limited to minor visual adjustments such as icons and text in the displayed tab.

For proxy-based phishing, various tools and frameworks are available for an attacker to build on. In contrast, no such tools which are explicitly developed for BitM phishing are currently known. However, our work supports the hypothesis of [19] that existing software can be converted into a BitM phishing tool.

### 6.3 Feasibility

We demonstrated that Browser-in-the-Middle is significantly more resource-demanding than other types of phishing, which makes the method unsuitable for simultaneous usage with many targets. However, most phishing campaigns run in a short time frame: [13] and [16] show that after about 10 hours after the first phishing email was sent, 75% of the victims have already visited the page. Right before a page gets listed as a phishing page, there is a concentration of connections to the phishing site. Therefore, a phishing campaign using BitM would require a vast amount of resources to be allocated for a short period of time in order to establish many remote desktop connections simultaneously. Thus our work confirms the claim by [19] that BitM is more suitable for use with few targets such as in spear phishing.

As demonstrated with the implementation of BitM using *gacamole*, the resource demand that arises with many sessions at the same time can be distributed over several systems. This would require many systems, each with enough performance to enable a remote desktop connection. Required resources incur costs. Attackers often act for monetary reasons and thus think primarily in economic terms. Accordingly, the use of BitM seems contradictory to the main goal of attackers.

Finally, it should be noted that none of the variants of BitM implemented here enabled use on mobile devices. According to a report from 2011 [2], back then almost one-fifth of all successful phishing attacks were attributed to users who used a mobile device. The actual percentage today may be higher, but even the exclusion of more than one fifth of all potential victims in advance argues against the use of BitM in broad practice.

### 6.4 Mitigation

After conceptualising, implementing and evaluating Browser-in-the-Middle in different ways, we propose four forms of mitigation.

**6.4.1 JavaScript restriction.** Both of the implemented and investigated variants of BitM depend on JavaScript, both for displaying the content as well as for capturing and sending user input. If JavaScript is prevented from being available for the phishing page, the web client will not work as intended and the attack will fail.

A similar approach to restricting JavaScript is the one proposed by [19] to disable WebSocket connections or at least to issue a warning in case of a established connection. However, we cannot generally recommend this approach for BitM, since implementations of BitM exist, that do not require a websocket connection, such

as the variant we implemented using RDP. Using HTTP would then only impair performance, but does not make an attack impossible.

**6.4.2 Use of browser-connected password managers.** Password managers, as our tests show, are another effective measure against BitM, if the page visited by the user is automatically recognized by the program used or the credentials are automatically filled in. This is the case when either a password manager built into a browser or a separately running application connected to the browser by a corresponding plug-in is used. While automated detection of the web page also protects against other phishing techniques, automatic selection of the correct HTML elements and insertion of password and user ID is specifically effective against BitM, because the HTML elements necessary for automatic recognition are missing when the content is displayed using a screen transmission. Without automatic recognition of the website, that means if the password is entered manually or the clipboard is used, password managers do not protect against BitM. As our tests show, it must be assumed that the clipboard also works in a BitM attack.

**6.4.3 Use of resistant authentication factors.** According to our test results, the most effective measure to prevent BitM phishing attacks is to use resistant factors in multi-factor authentication. In our tests, out of the currently most widely used variants of MFA, only the variant using a security key with WebAuthn and FIDO2 was able to reliably prevent a BitM attack. Software- and hardware-based variants are both equally effective against a BitM attack. In our tests, an authentication attempt using WebAuthn failed because both web clients of the implemented BitM variants did not support the passthrough of security tokens. But also for a significantly more complex implementation of a web client that could forward a request for such devices from the victim's browser to the attacker's browser, the WebAuthn standard would prevent a successful authentication, since the relying-party-id wouldn't match stored credentials.

**6.4.4 Browser Fingerprinting.** Another valid mitigation of BitM attacks, introduced as a valuable contribution by our reviewers, is the use of device- or browser fingerprinting as described by [6]. The information browser leak through API's, e.g., the used hardware, can be used to uniquely distinguish them. If this data is linked to an account, the request for authentication of an attacker could be denied, since its browser fingerprint would not match with the linked one. An attacker would need to passthrough the information from its victims browser to the authenticating server, which would greatly increase the expense of implementation.

## 7 CONCLUSION

Our experiments show that BitM enabled successful phishing for 12 out of 13 targeted web services with multi-factor authentication. This method therefore excels in attacking logins protected with more than one factor. Especially when compared to other common forms of phishing, such as using a proxy, a BitM attack is significantly more resistant to server-side countermeasures, as it is more difficult to detect such an attack from the target server's point of view. Our study shows that in terms of implementation expenses, a remote desktop infrastructure, implemented once, allows almost instant adaptation to newly targeted or changed web services. The

resource requirements of a BitM attack are several times higher compared to a conventional phishing method, and the fact that mobile users – a non-negligible proportion of potential victims – can already be excluded in advance due to their platforms, make it clear that a scaled use of the method to a large number of victims is rather unlikely. BitM seems to be more suited for targeted spear phishing attacks.

The most reliable way to mitigate BitM is the use of resistant factors for authentication, such as security keys, whose effectiveness is, however, undermined in practice by the continuous implementation of fallback factors. Based on our findings and previous knowledge, we submitted a novel entry for CAPEC, which got accepted by MITRE as the 701th attack pattern [11].

## REFERENCES

- [1] Tessian Phishing Statistics: Updated 2022. 2022. <https://www.tessian.com/blog/phishing-statistics-2020/>
- [2] Wade Baker, Mark Goudie, Alexander Hutton, C David Hylender, Jelle Niemantsverdriet, Christopher Novak, David Ostertag, Christopher Porter, Mike Rosen, Bryan Sartin, et al. 2011. 2011 data breach investigations report. *Verizon RISK Team*, Available: [www.verizonbusiness.com/resources/reports/rp\\_databreach-investigationsreport-2011\\_en\\_xg.pdf](http://www.verizonbusiness.com/resources/reports/rp_databreach-investigationsreport-2011_en_xg.pdf) (2011), 45.
- [3] Kang Leng Chiew, Kelvin Sheng Chek Yong, and Choon Lin Tan. 2018. A survey of phishing attacks: Their types, vectors and technical approaches. *Expert Systems with Applications* 106 (2018), 1–20. <https://doi.org/10.1016/j.eswa.2018.03.050>
- [4] Rachna Dhamija, J Doug Tygar, and Marti Hearst. 2006. Why phishing works. In *Proceedings of the SIGCHI conference on Human Factors in computing systems*. 581–590.
- [5] Timothy Dougan and Kevin Curran. 2012. Man in the browser attacks. *International Journal of Ambient Computing and Intelligence (IJACI)* 4, 1 (2012), 29–39.
- [6] Peter Eckersley. 2010. How Unique is Your Web Browser?. In *Proceedings of the 10th International Conference on Privacy Enhancing Technologies* (Berlin, Germany) (PETS'10). Springer-Verlag, Berlin, Heidelberg, 1–18.
- [7] CAPEC: Common Attack Pattern Enumeration and Classification. 2016. <http://capec.mitre.org>
- [8] The Apache Software Foundation FreeRDP. Last accessed 2023-01-31. <https://www.freerdp.com/>
- [9] K Gretchky. 2018. Evilginx-Advanced Phishing with Two-factor Authentication Bypass.
- [10] The Apache Software Foundation Guacamole. Last accessed 2023-01-31. <https://guacamole.apache.org>
- [11] CAPEC-701: Browser in the Middle (BitM). 2023. <https://capec.mitre.org/data/definitions/701.html>
- [12] Ankit Kumar Jain and Brij B Gupta. 2017. Phishing detection: analysis of visual similarity based approaches. *Security and Communication Networks* 2017 (2017).
- [13] Samuel Marchal, Jérôme François, Radu State, and Thomas Engel. 2012. Proactive discovery of phishing related domain names. In *Research in Attacks, Intrusions, and Defenses: 15th International Symposium, RAID 2012, Amsterdam, The Netherlands, September 12-14, 2012. Proceedings 15*. Springer, 190–209.
- [14] mr.d0x. 2022. Steal Credentials & Bypass 2FA Using noVNC. <https://mrd0x.com/bypass-2fa-using-novnc>
- [15] The Apache Software Foundation No Auth Extension. Last accessed 2023-01-31. <https://guacamole.apache.org/doc/0.8.3/gug/noauth.html>
- [16] Adam Oest, Penghui Zhang, Brad Wardman, Eric Nunes, Jakub Burgis, Ali Zand, Kurt Thomas, Adam Doupe, and Gail-Joon Ahn. 2020. Sunrise to sunset: Analyzing the end-to-end life cycle and effectiveness of phishing attacks at scale. In *USENIX Security Symposium*.
- [17] T. Richardson, Q. Stafford-Fraser, K.R. Wood, and A. Hopper. 1998. Virtual network computing. *IEEE Internet Computing* 2, 1 (1998), 33–38. <https://doi.org/10.1109/4236.656066>
- [18] Checkpoint Cyber Security. 2022. <https://blog.checkpoint.com/2022/04/19/social-networks-most-likely-to-be-imitated-by-criminal-groups-with-linkedin-now-accounting-for-half-of-all-phishing-attempts-worldwide/>
- [19] Franco Tommasi, Christian Catalano, and Ivan Taurino. 2022. Browser-in-the-Middle (BitM) Attack. *Int. J. Inf. Sec.* 21, 2 (2022), 179–189. <https://doi.org/10.1007/s10207-021-00548-5>
- [20] VadeSecure Phishers Favorites year-in review. 2022. <https://info.vadeseure.com/en/phishers-favorites-2021-year-in-review>